🧠 DevTrace v2 — Distributed Developer Observability Engine

🚀 1. Project Overview

DevTrace v2 is a high-performance, developer-centric observability platform designed to capture, analyze, replay, and introspect API traffic in real time.

Unlike traditional logging tools, DevTrace operates as an **inline programmable proxy layer** powered by:

- Event-driven architecture
- Real-time streaming
- Plugin-based extensibility

It enables developers to debug and simulate system behavior with **production-grade fidelity**, similar to internal tools used by companies like Stripe, Vercel, and Cloudflare.

⚡ 2. Core Architectural Upgrades

🧾 Event Sourcing (Immutable Logs)

- Every request-response cycle is stored as an **event**
- Append-only log → no mutation
- Enables:
  - Time-travel debugging
  - Full replayability
  - Deterministic debugging

🔀 CQRS (Command Query Responsibility Segregation)

Layer	Responsibility
Command	Capture & store events
Query	Read, analyze, visualize

👉 Write and read paths scale independently

🔌 Plugin-Based Processing Engine

Extensible system using Rust traits:

```
traitPlugin {
fnprocess(&self,event:&TraceEvent);
}
```

Examples:

- latency analyzer
- error detector
- auth validator
- data redactor

🌐 Real-Time Streaming Layer

- Streams events via WebSockets / gRPC
- Enables:

- live dashboards
- alerts
- monitoring

🧠 Local-First, Distributed-Ready

- Works as a **CLI tool locally**
- Can scale to:
  - multi-node ingestion
  - centralized aggregation

🏗️ 3. High-Level Architecture

```
Client (Browser / Postman)
        |
        ▼
DevTrace Proxy (Rust)
        |
 ┌──────┼──────────────┐
 ▼      ▼              ▼
Capture  Plugin Engine  Stream Engine
        |
        ▼
Event Bus (Kafka / NATS)
        |
 ┌──────┼──────────────┐
 ▼      ▼              ▼
Event Store   Dashboard   CLI Engine
        |
        ▼
Replay Engine
```

🧩 4. Sub-System Deep Dives

🔁 Proxy Engine (Core)

Responsibilities:

- Intercept HTTP requests
- Forward to target server
- Capture request & response

Flow:

```
Request → Capture → Forward → Capture Response → Emit Event
```

Built with:

- `tokio`
- `hyper`

📦 Event Model

```
structTraceEvent {
    request:RequestLog,
```

```
    response:ResponseLog,
    latency_ms:u128,
    timestamp:u64,
}
```

👉 This is your **single source of truth**

🧠 Plugin Engine

Processes events asynchronously:

- Runs in parallel
- Stateless or stateful

Examples:

- detect slow APIs (>500ms)
- detect 5xx errors
- redact sensitive headers

📊 Real-Time Dashboard

Stack: Next.js + WebSockets

Features:

- live request stream
- filtering & search
- latency graphs
- error heatmaps

🔁 Replay Engine (Key Feature)

Replay past requests:

```
devtrace replay--route /api/login
```

Capabilities:

- resend requests
- simulate failures
- debug nondeterministic bugs

📚 Event Store

Mode	Storage
Local	JSON / SQLite
Production	PostgreSQL / ClickHouse

Properties:

- append-only
- indexed by route & timestamp

⚡ Streaming Layer

Mode	Technology
Local	Tokio channels
Distributed	Kafka / NATS

🧬 5. Advanced Capabilities

🌍 Distributed Mode

```
Node A → Kafka → Aggregator → Dashboard
Node B → Kafka → Aggregator → Dashboard
```

🧠 Intelligent Analysis Layer

- anomaly detection
- traffic pattern clustering
- error prediction (future ML)

🔌 Plugin Ecosystem (Future)

```
implPluginforCustomAnalyzer {
fnprocess(&self,event:&TraceEvent) {
// custom logic
    }
}
```

🔐 Security Layer

- redact sensitive headers
- detect PII
- secure storage

🧰 6. Technology Stack

Core

- Rust
- Tokio
- Hyper / Reqwest

Streaming

- Local: Tokio channels
- Distributed: Kafka / NATS

Storage

- Dev: JSON / SQLite
- Prod: PostgreSQL / ClickHouse

Frontend

- Next.js
- WebSockets

Observability

- tracing (Rust)
- OpenTelemetry (optional)

🧠 7. Engineering Challenges

Ownership & Memory

- handling request bodies safely
- avoiding unnecessary cloning

Concurrency

- async request handling
- parallel plugin execution

System Design

- modular architecture
- event-driven thinking

Extensibility

- trait-based plugin system

🧨 8. Why This Project Matters

This is not:

- ❌ CRUD app
- ❌ tutorial project

This is:

> ✅ a **developer infrastructure tool**

Comparable to:

- Stripe internal debugging tools
- Vercel observability systems
- Datadog (dev-focused version)

🎯 9. Execution Roadmap

Phase 1 (Now)

- RequestLog struct
- ResponseLog struct
- Basic proxy
- Console logging

Phase 2

- latency tracking
- structured logging
- error handling

Phase 3

- event storage
- CLI querying
- filtering

Phase 4

- replay engine
- real-time streaming

Phase 5

- dashboard (Next.js)
- plugin system
- distributed mode